8.Memoria dinamica

La memoria dinamica è una parte del programma con diverse proprietà, ed è il programmatore che decide quando crearla e quando rimuoverla dalla memoria.

Funzioni

La libreria standard prevedere le seguenti 4 funzioni per creare la creazione della memoria dinamica :

- allocare un nuovo blocco in memoria
  - malloc(), riserva un blocco di memoria contiguo la cui dimensione in byte è almeno pari alla dimensione, il contenunto del blocco allocato è ***indeterminato***
  - calloc() , riserva un blocco di memoria dove la dimensione in byte sia count x size , in altre parole il blocco è largo abbastanza per tenere un array di count elementi ,inizializza ogni singolo byte a 0
  - entrambe le funzioni ritornare un puntatore a void , chiamato anche puntatore senza tipo
- Ridimensionare un memoria già allocata
  - realloc() , usata per diminuire o incrementare la dimensione della memoria utilizzata,e gli assegna un nuovo blocco di size type restituendogli l'indirizzo
    - è possibile passargli un puntatore nullo come argomento ptr, se lo facciamo la funzione diventa simile al `malloc()`
    - ritorna un puntatore nullo se non riesce a allocare una memoria con la grandezza richiesta , in questo caso non rilascia ne il blocco di memoria ne altera il contenuto di essa
  - free(), viene usata quando non abbiamo piu bisogno di una determinata memoria allocata

Una memoria allocata rimane riservata fino a quando il programma non la rilascia esplicitamente usante o `free()` o `realloc()`

realloc()

questo preserva il contenuto originale del blocco di memoria , non perde i dati vecchi quando ridimensiono

errore tipico

- **double free** = chiamare due volte free() causa un crash o comportamento indefinito del programma
- **Segmention fault** = accade quando un dereferenziamo un puntatore NULL o se accediamo a memoria che non è stata allocata

Deallocation

Dobbiamo sempre deallocare gli oggetti con free quando non abbiamo più bisogno di loro, altrimenti il puntatore potrebbe diventare irraggiungibile.
Se questo si ripete , lo spazio può esser altamente consumato :

- quando è altamente consumato il programma puo crashare o bloccarsi o non è possibile allocare ciò che si vuole
- È un prob se il programme viene seguito per un lungo periodo

Dangling pointers

I puntatori pendenti (*dangling pointers*) si verificano durante la distruzione di un oggetto, quando un oggetto che ha un riferimento in ingresso viene eliminato o deallocato senza che il valore del puntatore venga modificato, lasciando il puntatore a puntare ancora alla posizione di memoria dell'oggetto deallocato